

LocalAI TV — Database & Storage Flow, Explained Simply

A plain-language walkthrough of how content moves, where it is stored, and how it reaches the live screen

Prepared for: Koneti Mohan Reddy (Managing Director) **Prepared by:** LocalAI TV Architecture Team
Date: 18 May 2026 **Purpose:** A clear, non-technical understanding of the full data & storage structure — so you can review it, discuss internally, and decide on folder/flow/scalability changes **before** Phase-1.
Reading level: No coding knowledge needed. Technical names are shown only so your AI team recognises them; every one is explained in plain words.

⚠ First — what is LIVE today vs. what is DESIGNED & waiting

So you are never misled, here is the honest status:

Part	Status
The mobile app (8 upload forms, 9 channels, local pages)	✅ Live — you are reviewing it now
The 9 constituency channels (Kurnool, Tirupati, etc.)	✅ Live (currently served via YouTube bulletins)
Admin Dashboard (roles, moderation, 15 modules)	🟡 Visual demo inside the app — the <i>design</i> is approved; the real backend is not yet built
The two databases + AWS S3 storage + CDN	🟡 Designed & approved, NOT yet built — this is gated behind Phase-0 validation
AI pipeline database move to AWS	🟡 Planned (moving from Google Cloud → AWS, not done yet)

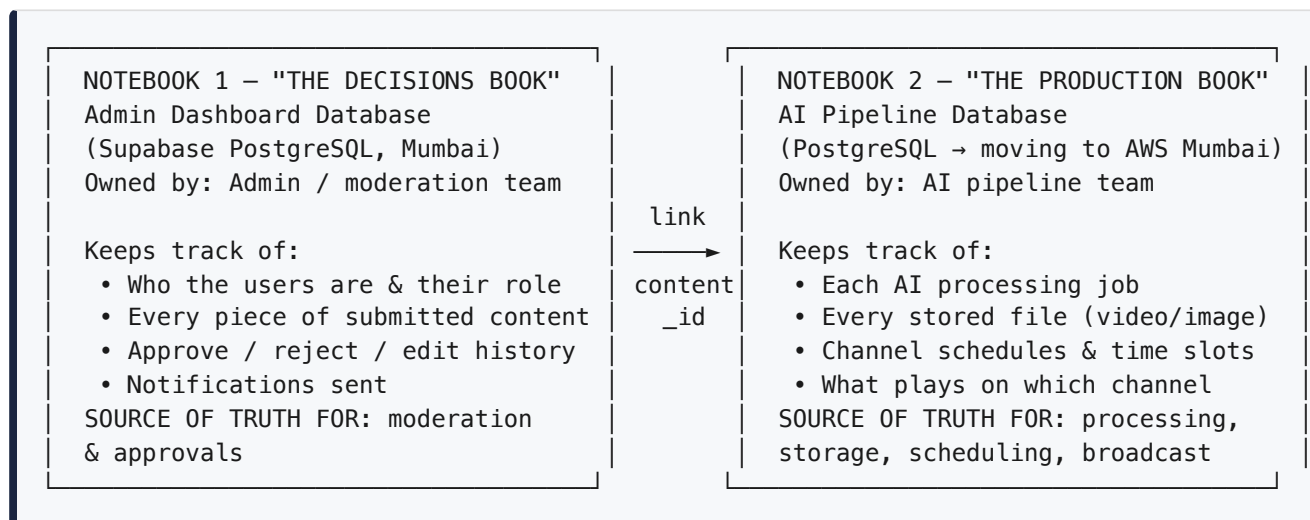
This document explains the **approved target structure**. It is **documentation only** — nothing here is being built or executed. The Phase-0 gate remains held; only you lift it.

One refinement to note: the original v1.3 plan named *Cloudflare* for storage/CDN. The Architecture Team's accepted correction (recorded in Addendum A, 18 May) replaces this with **AWS S3 + AWS CloudFront** — to protect your already-live Play Store app. This document reflects the **corrected AWS design**, which is the current truth.

1. Database Structure Overview

The simplest way to picture it: there are TWO notebooks, not one

LocalAI TV deliberately keeps **two separate databases**. Think of them as two notebooks owned by two different teams:



Why two and not one?

Reason	In plain words
Separation of duties	The admin team owns "decisions"; the AI team owns "production". Neither disturbs the other.
Very different sizes	The Decisions book is small (a few thousand decisions/day). The Production book is huge (hundreds of thousands of jobs/day at full scale).
Safety	If the Decisions book is briefly down, the AI Production book keeps working on its queue. One problem does not stop everything.
Team independence	Each team can improve its own notebook without breaking the other's.

The single link between them: every piece of content gets one unique ID number (`content_id`) the moment it is submitted. That same ID travels with the content everywhere. That is the *only* shared key — it lets us trace any item end-to-end across both notebooks.

Module-wise structure (what each notebook records)

Notebook 1 — Admin / Decisions keeps these record-sets ("tables"):

- **Users** — every reporter, admin, and citizen, with their role (Super Admin / Master Admin / Admin / Citizen), assigned area, and login security.
- **Content** — every submission: who sent it, which state/district/constituency, which category, the form details, the media files, and its current status (pending → approved/rejected → ready).
- **Audit log** — an unchangeable history of every approve/edit/reject and who did it.
- **Webhook deliveries** — proof that an approved item was successfully handed to the AI pipeline.
- **Notifications** — messages sent back to the uploader.

Notebook 2 — AI Pipeline / Production keeps these record-sets:

- **Processing jobs** — the life of each job (queued → processing → done/failed).
- **Content assets** — a row for **every single file** stored (input image, generated voice, final video, thumbnail), with its exact storage location and checksum.
- **Collections** — logical groupings (e.g. "Kurnool birthdays", "Tirupati news bulletin") so the scheduler asks for content *by meaning*, never by hunting through folders.
- **Schedule tables** — the channel programme, the daily time-slots, and manual overrides.
- **Channel config & display rules** — per-channel filtering and auto-expiry/rotation rules (built to scale to thousands of channels).
- **Callbacks outbox** — proof that the AI pipeline reported finished work back to the admin side.

How data is categorised (the key idea)

Every item is filed by a **4-level address**, exactly like a postal address:

```
STATE → DISTRICT → CONSTITUENCY → CATEGORY
(AP/TG)  (e.g. Kurnool) (e.g. Kurnool)  (News / Birthday / Marriage / ...)
```

This single addressing scheme is used **everywhere** — in the database, in the storage folders, and in the channel filtering. That consistency is what makes the system scalable.

2. Reporter / Citizen-Contributor Upload Flow

There are **two kinds of uploader**, and the system treats them slightly differently:

Uploader	What they send	Path it takes
Citizen (ordinary user)	Birthday, marriage, shopping, event, job, rental, car, or a news tip — 1–3 files + details	Goes into the review queue first; an admin must approve before anything happens
Reporter / Admin-direct	Professional news/video shot by your team	Can be marked "raw video" and go on a faster track, still logged and scheduled

Step-by-step: what happens when content is uploaded

1. SUBMIT

User fills a form in the app (1–3 media files + headline + description + location).
→ A unique `content_id` is created.
→ Files are uploaded to safe storage (the "pending" area).
→ Automatic safety checks run: bot check, virus scan, duplicate check, and an unsafe-image (NSFW) check that flags risky items for priority review.
→ A record is written to the Admin database with status = "pending".

2. WAITS IN THE REVIEW QUEUE

The item now sits in the admin moderation queue, filed under its `state/district/constituency/category` address.

3. AN ADMIN PICKS IT UP (safe lock)

When an admin opens an item, it is locked to that admin for 30 minutes so two admins can never act on the same item by mistake.

4. ADMIN DECIDES

✅ Approve ✎ Edit then approve (the original is kept, read-only)
❌ Reject (reason recorded; uploader notified) ➔ Escalate to a senior admin

5. (Only if approved) HANDED TO THE AI PIPELINE

A reliable "webhook" message carries the `content_id` to the AI pipeline.
If the pipeline is briefly unreachable, it retries automatically (1s → 5s → 30s → 5min → 30min) and never silently drops the item.

6. AI PIPELINE PROCESSES IT

Summarise the text → generate Telugu voice → assemble the video.
The finished files are stored in the "ai-processed" area under the item's `state/district/constituency/category/content_id` address.

7. PIPELINE REPORTS BACK

It tells the Admin database "this `content_id` is ready", with the final video link and a thumbnail. Status becomes "ready for bulletin".

8. SCHEDULED & BROADCAST

The item is placed into a daily time-slot for its channel and goes into the live bulletin. (Birthday wishes are auto-placed on the birthday date.) The uploader gets a notification ~10 min before air.

Where files land at each stage (the storage buckets):

Stage	Storage location (folder)
Just uploaded, awaiting review	<code>citizen-uploads/pending/</code>
Approved by admin, queued for AI	<code>citizen-uploads/approved/</code>
Reporter / team raw video	<code>admin-direct-uploads/</code>
Finished AI item (final video, voice, thumbnail)	<code>ai-processed/<state>/<district>/<constituency>/<category>/<content_id>/</code>
Finished full bulletin	<code>ai-processed-bulletins/<state>/<district>/<constituency>/bul_<date_time>/</code>

3. Local Constituency Data Flow (Kurnool, Tirupati & others)

The 9 live channels today

Andhra Pradesh (5)	Telangana (4)
Kurnool · Guntur · Nellore · Kakinada · Tirupati	Khammam · Karimnagar · Warangal · Nalgonda

How content is kept separate per constituency

Because **every item carries its state → district → constituency address**, separation is automatic:

```

A citizen in Tirupati uploads a birthday wish
    |
    ▼
content_id created · address = AP / Tirupati / Tirupati / Birthday
    |
    ▼
Stored at: ai-processed/AP/Tirupati/Tirupati/birthday/<content_id>/
    |
    ▼
The "Tirupati" channel's schedule only pulls items whose
address = AP / ... / Tirupati → it can NEVER show Kurnool content

```

- **Display side (the app):** when a viewer picks "Tirupati", the app shows only that constituency's live channel, local pages, and classifieds — because it filters on the same constituency address. (This is the routing behaviour we already fixed in the app — the home page reads the chosen constituency, never a hard-coded one.)
- **Scale-ready:** the design uses pre-computed per-channel filters (so even at thousands of channels, the app does not slow down running live searches).

4. User Content Categories — where each kind goes

The app has **8 upload categories**. All follow the same storage rule, with small differences in how they are scheduled/displayed:

Category	What it is	Stored under	How it reaches the screen
 News	Local news / information	<code>ai-processed/.../news/</code>	AI-processed → placed in the news bulletin
 Birthdays	Birthday wishes	<code>ai-processed/.../birthday/</code>	Auto-scheduled for the birthday date , rotated
 Marriages	Weddings & anniversaries	<code>ai-processed/.../marriage/</code>	Scheduled to a slot; auto-expires after the event window
 Events	Local events / functions	<code>ai-processed/.../live_event/</code>	Scheduled around the event date
 Jobs	Job postings	<code>ai-processed/.../classified/ (job)</code>	Shown in the local classifieds + slot rotation
 Car / Motorcycle	Vehicle sale ads	<code>ai-processed/.../car_sale/</code>	Classifieds + rotation, auto-expiry
 Rentals	House / property rentals	<code>ai-processed/.../classified/ (rent)</code>	Classifieds + rotation, auto-expiry
 Shopping	Shop / business promotions & ads	<code>ai-processed/.../shopping/</code>	Classifieds / promo rotation

Public notices & advertisements ride on the same rails — a public notice is treated as a News/Notice item; a paid advertisement is a Shopping/promo collection with its own rotation rule. No separate machinery is needed; only a collection label differs.

For every category, the flow is identical: `Upload → safety checks → admin approval → AI processing → stored in ai-processed/<address>/<category>/<content_id>/ → scheduled → live`. The only differences are **scheduling rules** (e.g. birthdays land on the birthday date; classifieds auto-expire) — not storage location.

5. Storage Architecture (the cloud "filing cabinet")

Everything lives in **one AWS S3 bucket in Mumbai**: `localaitv-content-mumbai`. Think of the bucket as a large filing cabinet; the folders below are its labelled drawers.

```

s3://localaitv-content-mumbai/
├── citizen-uploads/
│   ├── pending/        ← just uploaded, awaiting admin review
│   └── approved/       ← admin-approved, waiting for AI
├── admin-direct-uploads/ ← reporter/team raw video (fast track)
├── notebooklm/          ← long-form/explainer source material
│   ├── individual-news/<state>/<district>/<constituency>/
│   ├── bulletins/ (district / state / national)
│   └── debates/<level>/<state>/<district>/<constituency>/
├── ai-processed/         ← FINISHED individual items
│   ├── <state>/<district>/<constituency>/<category>/<content_id>/
│   │   ├── final.mp4    (the video)
│   │   ├── headline.mp3 (the Telugu voice)
│   │   ├── thumb.jpg    (the thumbnail image)
│   │   └── meta.json    (the details record)
├── ai-processed-bulletins/ ← FINISHED full bulletins
│   ├── <state>/<district>/<constituency>/bul_<date_time>/
│   │   ├── final_bulletin.mp4
│   │   ├── bulletin_manifest.json
│   │   ├── thumbnail.jpg
│   │   ├── segments/ (item_001.mp4 ... item_NNN.mp4)
│   │   └── metadata.json

```

Video vs Image/Document flow

- **Videos** (final.mp4, bulletins): the heaviest files — these are what the CDN serves most, and what the cost-saving tiers target.
- **Images** (thumb.jpg) & **documents** (meta.json / manifests): small, kept close at hand for fast listing; manifests are kept long-term even when old videos are archived.

Temporary vs Permanent storage

- **Temporary:** a **temp/** working area is **auto-deleted after 1 day**; bulletin "segments" (the building blocks of a bulletin) are **auto-deleted after 7 days** once the final bulletin is built. This keeps junk from accumulating.
- **Permanent:** the final video, voice, thumbnail and the records are kept — but they automatically move to **cheaper storage as they age** (see below).

Backup & cost-saving (automatic ageing)

Old content is not deleted — it is moved to progressively cheaper storage automatically:

Age of content	Moves to	Roughly the cost
0–7 days (fresh)	Standard	full price
8+ days	Standard-Infrequent	~50% cheaper
~1 month+	Glacier Instant	~75% cheaper
~3 months+	Deep Archive	~95% cheaper

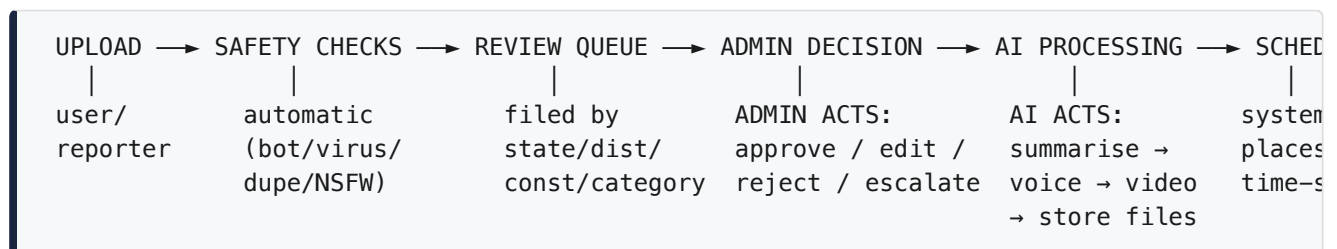
This single rule **saves 50–75% of storage cost at scale with no manual work** — old bulletins are still retrievable, just cheaper to keep. Manifests/records are kept even longer than the heavy videos, so history is never lost.

CDN (fast delivery + cost protection)

A CDN (**AWS CloudFront**) sits in front of the bucket, reachable at `content.localaitv.com`. It caches popular videos close to viewers, so ~90% of plays never touch the bucket — fast for users, and it **prevents catastrophic data-transfer bills at scale**. Cache rules: photos 7 days, AI videos 30 days, bulletins 24 hours, audio 30 days.

Note: v1.3 originally said *Cloudflare* for this layer; the accepted AWS correction uses **CloudFront** with a single **Hostinger CNAME**, so your live app's address (`aiservices.localaitv.com`) is never touched.

6. Processing Flow & Admin Involvement (one picture)



Who does what at each stage:

Stage	Human / system	Admin involved?
Upload	Citizen or reporter	No
Safety checks	Automatic	No (only flags risky items)
Review queue	System files it	No
Decision	Admin	Yes — the key human gate
AI processing	AI pipeline (automatic)	No
Report-back	Automatic	No
Scheduling	Automatic (rules + slots)	Admin can override a slot manually
Broadcast	Automatic	Audit log records what aired & when

AI processing in plain words: the AI reads the submitted text, writes a concise Telugu script, generates a natural Telugu voice, and assembles a video with the images/clips — then files the finished video in the right folder. The admin never has to touch the AI step; they only control **approve/reject** and **schedule overrides**.

7. Recommendations — additional folders, scalability, backup

These are the Architecture Team's suggestions for you to consider during your review:

7.1 Recommended additional folders (small, high-value additions)

Suggested folder	Why
<code>rejected-archive/<address>/</code>	Keep rejected items for a fixed period (e.g. 30 days) for dispute/audit, then auto-delete — instead of losing them immediately.
<code>advertisements/<address>/</code> (own collection)	Paid ads tracked separately from organic Shopping posts → cleaner billing & reporting later.
<code>public-notice/<address>/</code> (own collection)	Government/official notices kept distinct from general news for compliance and quick retrieval.
<code>legal-hold/</code>	A protected area where nothing auto-deletes — for any item under a complaint/legal request.
<code>exports/reports/</code>	Where periodic analytics/billing exports are written, separate from media.

7.2 Scalability planning

- **Keep the 4-level address everywhere** (state→district→constituency→category) — it is already the backbone; do not add parallel schemes.
- **Add `district` as a real filtering level now** (not just inside the path) so district-level bundles (e.g. "all Kurnool-district news") are cheap when you grow past a few hundred channels.
- **Pre-computed filters & collections are already designed in** — confirm the AI team keeps these (they are what allow thousands of channels without slow live searches).
- **Plan the database for growth:** the AI Production database should move to a high-availability setup (and, much later, an auto-scaling engine) **before** Phase-1 production traffic — already on the Phase-1 list.

7.3 District / constituency separation

- Today separation is by *folder path + filter*. Recommendation: also create a **per-constituency "collection" record** for each category so a new constituency can be switched on by adding a row, **without moving any files**. This makes onboarding a new town a 1-line change, not a storage reorganisation.

7.4 Archive & backup structure

- The automatic ageing tiers (above) already give cheap long-term retention. **Add: a daily database backup** kept 30 days and a **monthly snapshot** kept 1 year, for *both* notebooks (not just storage).
- Keep the **Google Cloud database read-only for 7 days** after the AWS move (rollback safety) and take a **final snapshot** before switching it off — already specified.
- **Recommended retention policy to decide internally:** finished videos kept hot 90 days → archived 1 year → deep-archive thereafter; manifests/records kept 3+ years for history. (Numbers are yours to set — this is the structure.)

8. Summary & what we need from you

In one paragraph: Content is uploaded → checked → an admin approves it → the AI turns it into a Telugu video → it is filed in one Mumbai cloud bucket under a clear *state/district/constituency/category* address → a CDN delivers it fast and cheaply → it is scheduled into the right constituency channel and broadcast. Two separate databases keep the "decisions" and the "production" cleanly apart, linked by one ID. Old content ages automatically into cheaper storage so it is kept without high cost.

What we need from you (your review feedback):

1. Which of the **suggested extra folders** (§7.1) you want — especially separate **Advertisements** and **Public Notices**.
2. Your preferred **retention periods** (how long to keep videos hot vs. archived).
3. Any **administrative structure** changes (who approves what at Super/Master/Admin level).
4. Confirmation that **district-level grouping** (§7.2) should be added before Phase-1.

After your feedback, the accepted points are recorded in **Addendum A** and applied as v1.4 — but only **after the Phase-0 gate is lifted by you**. Nothing here changes the frozen v1.3 or starts any build.

Documentation only · No execution · Phase-0 gate remains held · Reflects the accepted AWS S3 + CloudFront design (Addendum A) over the original v1.3 Cloudflare wording. **LocalAI TV Architecture Team** · LocalAI Media Network Pvt Ltd · CIN: U63910KA2025PTC212593 · Hyderabad, India